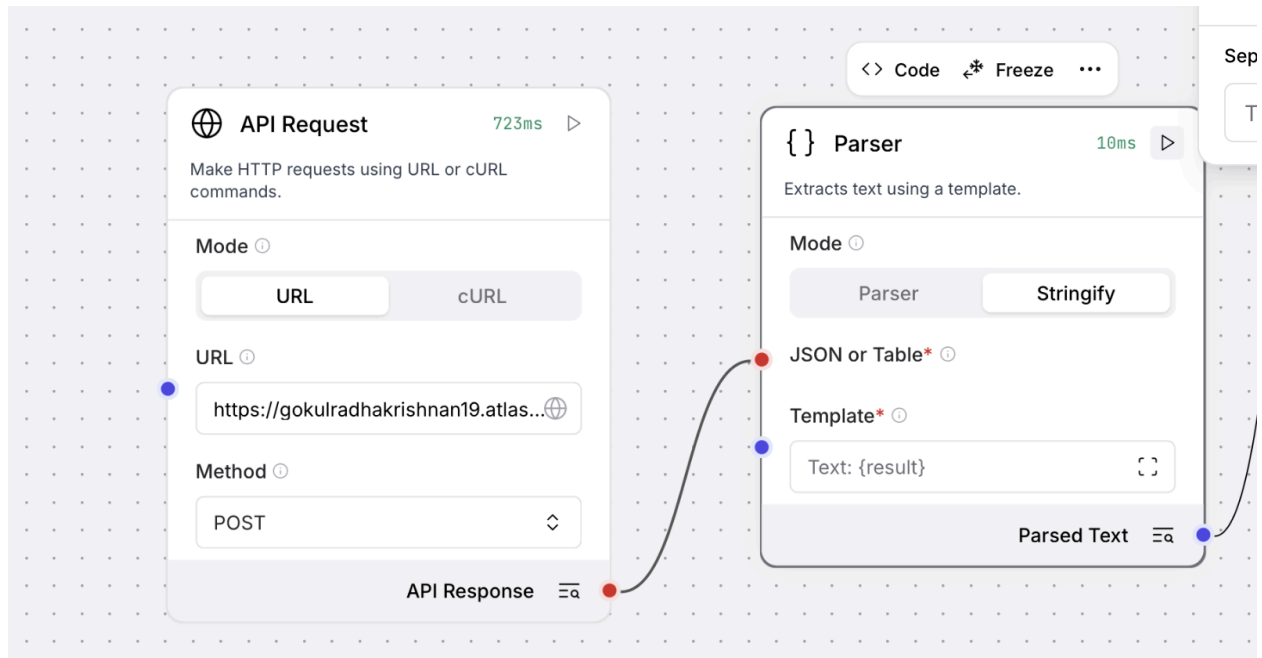


Week 6 Day-1 Assignment-1

A) User Story Review Agent With Langflow:

Api request connect to Parser:



Parser Output:

Component Output

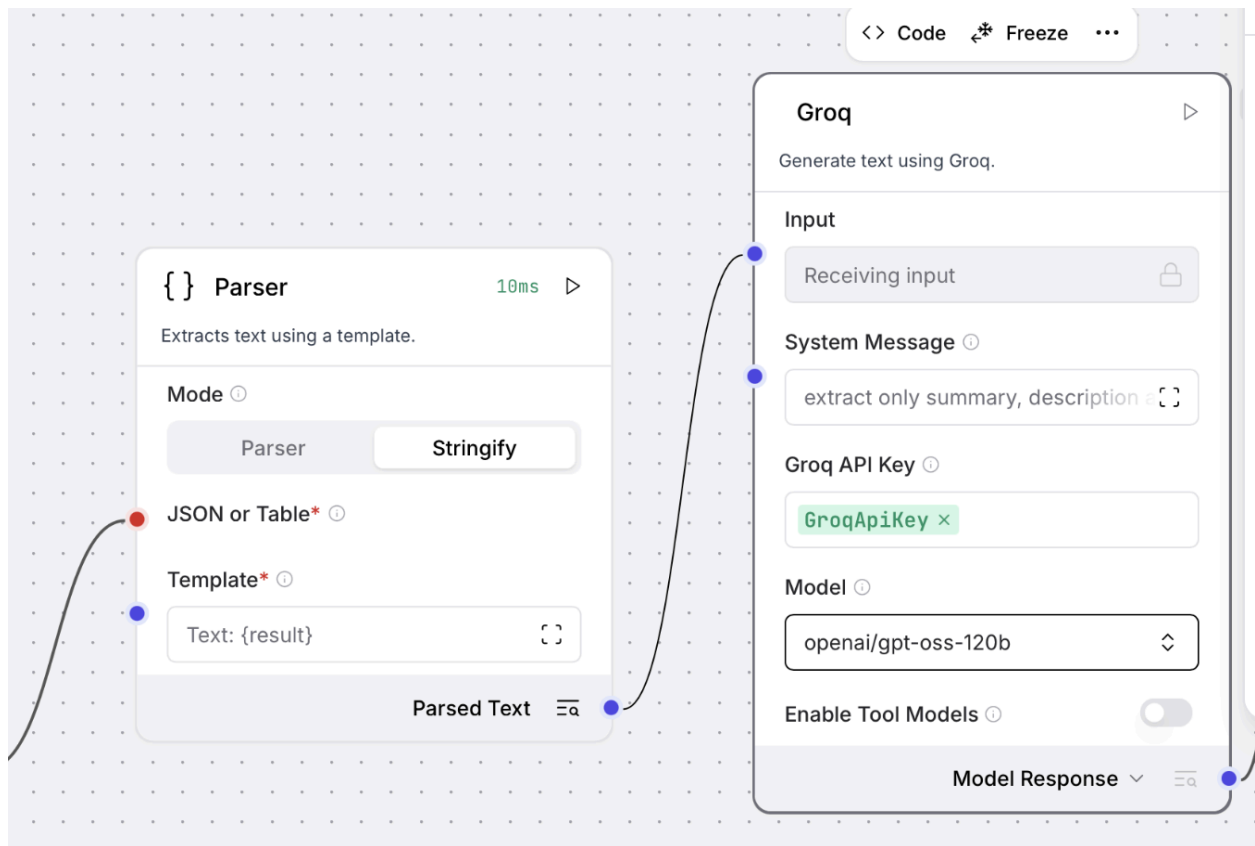
Inspect the output of the component below.

Outputs Logs

```
'''json
{
  "source": "https://gokulradhakrishnan19.atlassian.net/rest/api/3/search/jql",
  "status_code": 200,
  "response_headers": {
    "content-type": "application/json;charset=UTF-8",
    "transfer-encoding": "chunked",
    "connection": "keep-alive",
    "date": "Wed, 06 May 2026 13:39:14 GMT",
    "server": "AtlassianEdge",
    "timing-allow-origin": "*",
    "x-arequestid": "be17e050d9b1de5f83c554cd34e41949",
    "set-cookie": "atlassian.xsrf.token=2ab52767cfc9727d57d4335d9e631d41cde6076b_lin; Path=/; SameSite=None; Secure",
    "x-aaccountid": "64250e6b67102fc717c26f0e",
    "cache-control": "no-cache, no-store, no-transform",
    "x-ratelimit-limit": "200",
    "x-ratelimit-remaining": "199",
    "vary": "Accept-Encoding",
    "content-encoding": "gzip",
    "x-content-type-options": "nosniff",
    "x-xss-protection": "1; mode=block",
    "atl-traceid": "cf0f425512454d12a7a6cd62c0bbad04",
    "atl-request-id": "cf0f4255-1245-4d12-a7a6-cd62c0bbad04",
    "strict-transport-security": "max-age=63072000; includeSubDomains; preload",
    "report-to": "{\n  \"endpoints\": [\n    {\n      \"url\": \"https://dz8aopenkv6s.cloudfront.net/\", \"group\": \"endpoint-1\", \"include_subdomains\": true, \"max_age\": 600\n    }\n  ],\n  \"failure_fraction\": 0.01, \"include_subdomains\": true, \"max_age\": 600, \"report_to\": \"endpoint-1\" }\n",
    "nel": "{\n  \"failure_fraction\": 0.01, \"include_subdomains\": true, \"max_age\": 600, \"report_to\": \"endpoint-1\" }\n",
    "x-cache": "Miss from cloudfront",
    "via": "1.1 8b6dd0d472e21058db2ca2ab69bddafa.cloudfront.net (CloudFront)"
}
```

Close

Parser to Groq:



Groq Output:

Component Output

Inspect the output of the component below.

Outputs

Logs

```
**Extracted fields**  
  
'''json  
{  
  "issues": [  
    {  
      "key": "TH-1",  
      "summary": "Consultant Dashboard My Inpatients",  
      "description": "As a user, I want to consultant dashboard my inpatients so that the related functionality works as expected."  
    }  
  ]  
}  
...  
'''
```

Close

Groq to prompt template:

The image shows a workflow in the Groq interface. On the left, the 'Groq' panel is configured with the following settings:

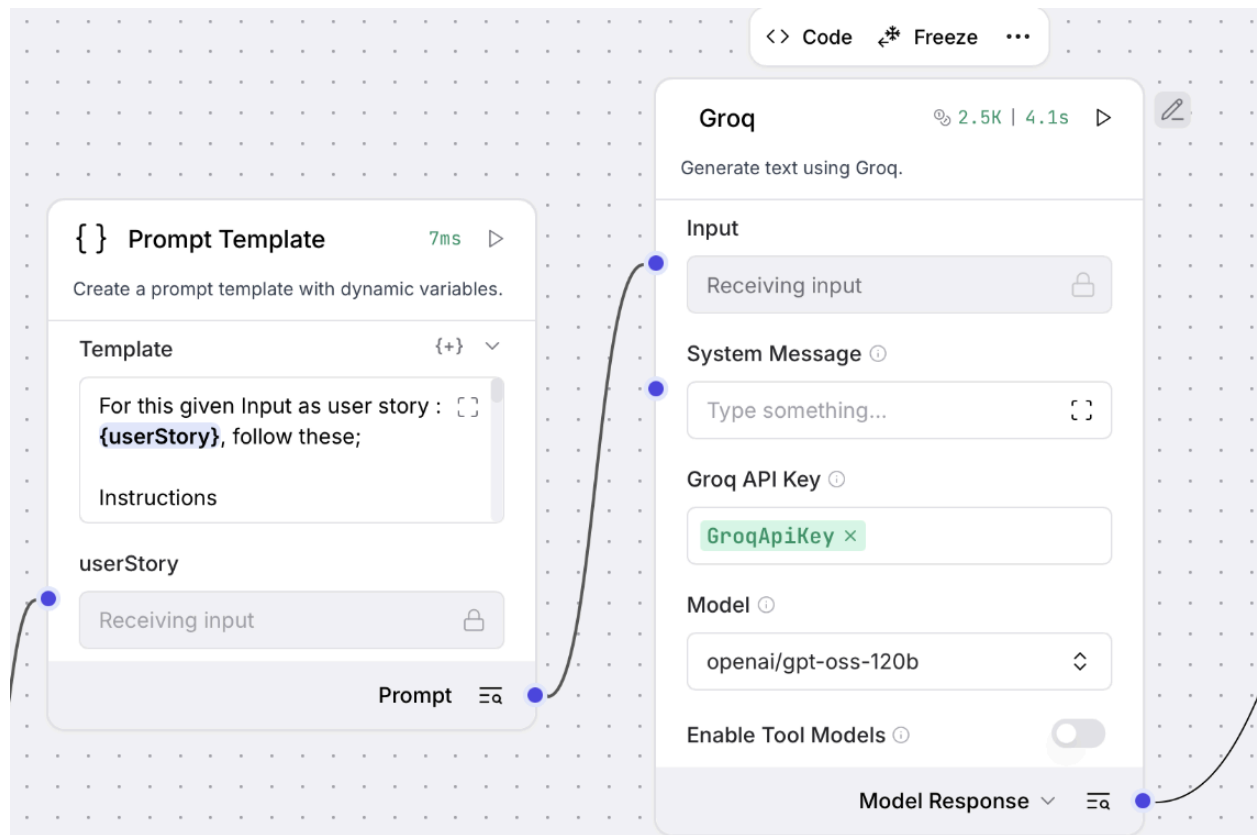
- Input:** Receiving input
- System Message:** extract only summary, description
- Groq API Key:** GroqApiKey
- Model:** openai/gpt-oss-120b
- Enable Tool Models:** Disabled

On the right, the 'Prompt Template' panel is configured with the following settings:

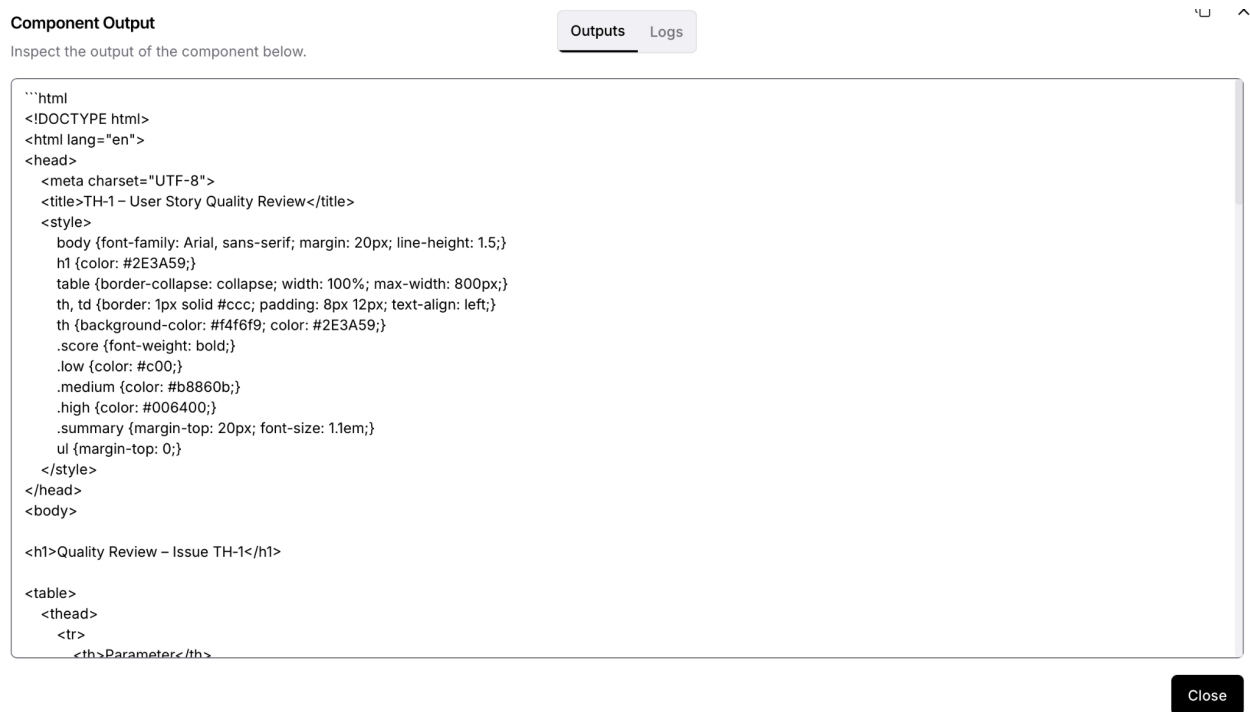
- Template:** For this given Input as user story : {userStory}, follow these;
- Instructions:** (Empty)
- userStory:** Receiving input
- Prompt:** (Expanded view)

Arrows indicate the flow of data: from the 'Input' field in the Groq panel to the 'userStory' field in the Prompt Template panel, and from the 'Prompt' field in the Prompt Template panel to the 'Model Response' field in the Groq panel.

Prompt to Groq again:



Html output from Groq:



Custom component created by modifying the python code to include html option:

Write File Gokul HTML

Save Data, DataFrame, or Message to file including HTML reports.

Input*

File Name*

US_Review_1

File Format

html

Append

File Path



Write File Gokul HTML



Save Data, DataFrame, or Message to file including HTML reports.

Input*

File Name*

US_Review_1



File Format

html



Search options...

csv

excel

json

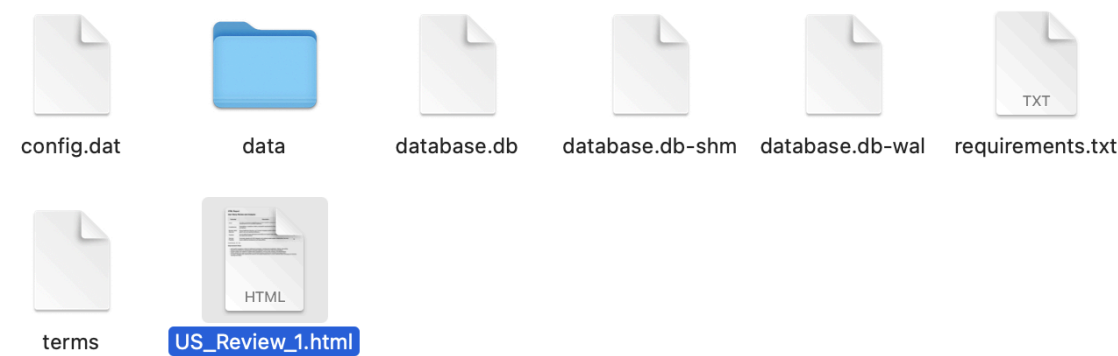
markdown

html



txt

Html output created in specified path:



Output:



HTML Report

User Story Review and Analysis

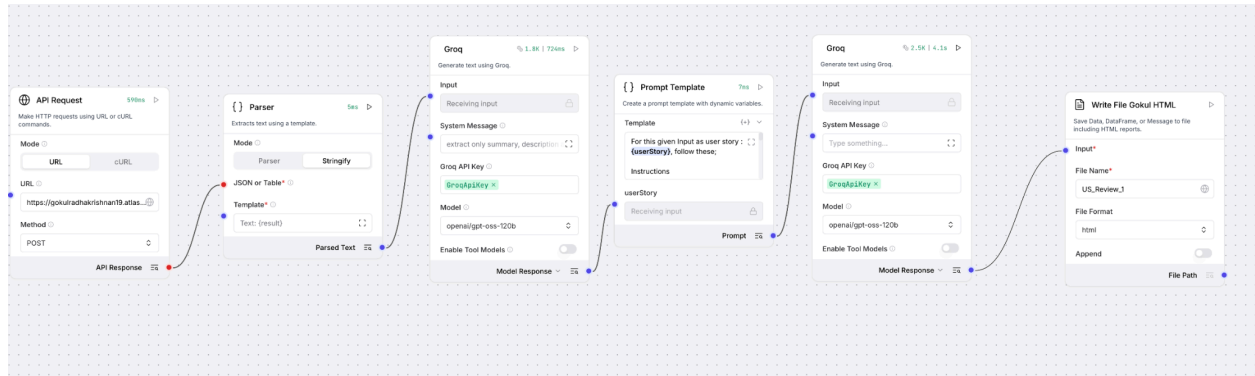
Parameter	Description	Score (out of 20)
Clarity	The story is somewhat straightforward but lacks detailed acceptance criteria and specific requirements for the consultant dashboard.	12
Completeness	Preconditions, acceptance criteria, and specific requirements for the consultant dashboard are not well-defined.	10
Business Value Alignment	Strong healthcare relevance, as it involves managing inpatient information, which is crucial for patient care and operational efficiency.	18
Testability	Can be tested through dashboard functionality and inpatient data management, but lacks specific test cases and scenarios.	14
Technical Feasibility	Technically feasible with EHR integration and existing hospital system infrastructure, but may require additional development and testing efforts.	16

Overall Score: 70 / 100

Improvement Areas:

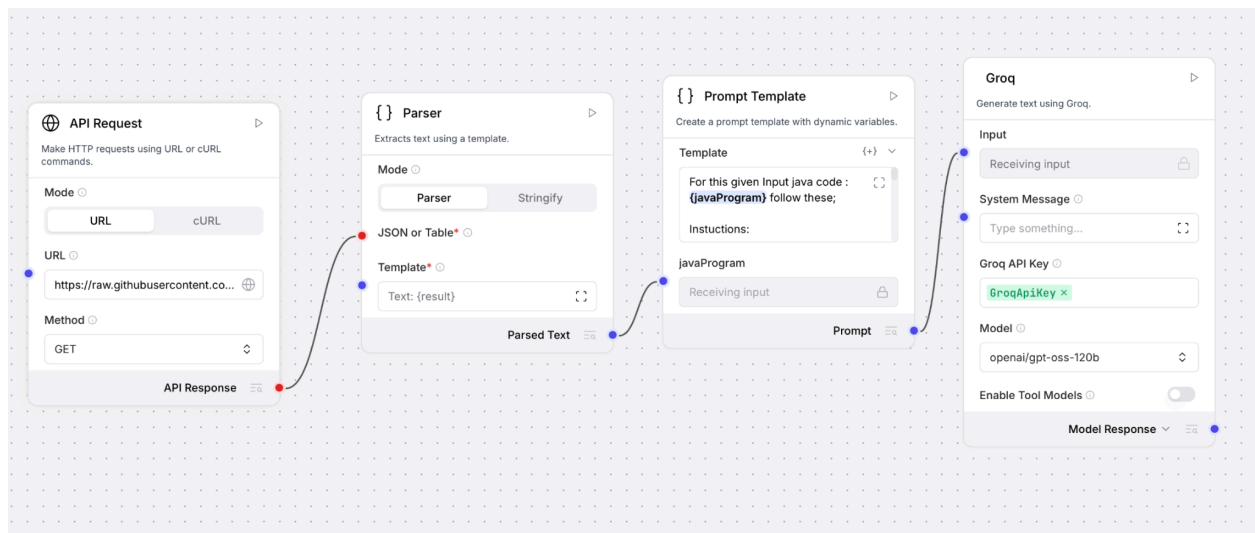
- Add explicit acceptance criteria for dashboard functionality, including data visualization, filtering, and sorting.
- Specify preconditions for accessing the consultant dashboard, such as user roles and permissions.
- Include system actor details for inpatient data management, such as nurses, doctors, and administrators.
- Define specific test cases and scenarios for dashboard testing, including error handling and edge cases.
- Consider integrating with existing EHR systems and hospital infrastructure to ensure seamless data exchange and minimize development efforts.

Entire Langflow:



B) Code Review Agent With Langflow:

Entire successful langflow:



Output from Groq:

Component Output

Inspect the output of the component below.

Outputs

Logs

```
**Code Review – LearnNestedFrame.java**
*(Selenium-based UI automation snippet)*

| Parameter | Description (key observations) | Score (out of 20) |
|-----|-----|-----|
| **Code Readability** | • Naming follows Java conventions ('LearnNestedFrame', 'driver', 'element'). <br>• Indentation is consistent and the code is short enough to follow visually. <br>• No Javadoc or inline comments explaining *why* the frames are switched, what the test validates, or any pre-conditions. <br>• The XPath string is escaped and a bit hard to read; a constant or a descriptive variable name would help. | **14** |
| **Code Efficiency** | • No loops or heavy processing – the snippet is already O(1). <br>• Uses an implicit wait of 15 s, which can mask performance problems and slow down the test suite. <br>• Switching to frames by index ('frame(0)') is brittle; locating by a stable attribute would be more reliable. | **15** |
| **Error Handling** | • No 'try-catch' / 'finally' block; any 'NoSuchElementException', 'TimeoutException', or driver-initialisation failure will abort the JVM. <br>• No logging or meaningful error messages. <br>• The driver is never closed ('driver.quit()'), leading to orphaned browser processes. | **5** |
| **Maintainability** | • All logic lives in a single 'main' method – not reusable or extensible. <br>• Hard-coded URL, locators, and frame indexes make future changes error-prone. <br>• No Page Object Model (POM) abstraction, no test framework (TestNG/JUnit) integration, and no configuration handling. | **8** |
| **Best Practices & Standards** | • Missing **SOLID** considerations – the class does a single thing (launch & click) but mixes driver lifecycle, navigation, and test actions. <br>• No use of **DRY** – the same driver setup would be duplicated across tests. <br>• No explicit 'System.setProperty' for the Chrome driver binary (relies on WebDriverManager or system PATH). <br>• No unit/integration test scaffolding, no assertions, and no reporting. <br>• Imports are correct but could be trimmed if a test framework is introduced. | **6** |

### Overall Score
**48 / 100**
```

Component Output

Inspect the output of the component below.

Outputs

Logs

```
## Actionable Feedback & Recommendations

| Area | Recommendation | Why it matters |
|-----|-----|-----|
| **Driver Lifecycle** | Wrap driver usage in a 'try-finally' (or try-with-resources) block and call 'driver.quit()' in the 'finally'. | Guarantees browser cleanup, prevents resource leaks on failures. |
| **Exception Management** | Catch Selenium-specific exceptions ('NoSuchElementException', 'TimeoutException') and log a clear message (use SLF4J/Log4j). | Improves debuggability and allows the test suite to continue gracefully. |
| **Wait Strategy** | Replace the global implicit wait with explicit 'WebDriverWait' for the specific elements/frames you interact with. | Reduces unnecessary waiting, makes failures faster and more deterministic. |
| **Locators & Frame Switching** | Store locators as 'private static final By' constants or use a Page Object. Prefer locating frames by a stable attribute ('id', 'name') instead of index. | Improves readability, eases maintenance when UI changes, and reduces flakiness. |
| **Modularisation** | Extract the following responsibilities into separate methods or classes: <br>1. 'setUpDriver()' – driver init & config <br>2. 'navigateToPage()' – URL loading <br>3. 'switchToNestedFrame()' – frame handling <br>4. 'clickButton()' – action <br>5. 'tearDown()' – quit driver. | Encourages reuse, simplifies future extensions (e.g., adding more test steps). |
| **Test Framework Integration** | Convert the 'main' method into a TestNG/JUnit test method annotated with '@Test'. Use '@BeforeClass/@AfterClass' for setup/teardown. | Enables parallel execution, reporting, and CI/CD integration. |
| **Configuration Management** | Externalise URL, timeouts, and driver binary path to a properties/YAML file or environment variables. | Allows the same code to run across environments (dev, QA, prod) without code changes. |
| **Documentation** | Add Javadoc for the class and public methods, and inline comments describing the purpose of each frame switch. | Helps new team members understand intent quickly. |
| **Logging** | Introduce a logger ('private static final Logger LOG = LoggerFactory.getLogger(LearnNestedFrame.class);') and log key steps (driver start, navigation, frame switches, click action). | Provides runtime visibility and aids troubleshooting. |
| **Static Analysis & Formatting** | Run tools like SpotBugs, Checkstyle, and Google Java Format to enforce coding standards automatically. | Guarantees consistent style across the repository. |

## Summary

The submitted snippet demonstrates a working Selenium flow for interacting with a nested frame, but it is far from production-ready. While the code is syntactically correct and readable at a glance, it lacks the structural, defensive, and configurational qualities expected in a shared automation repository.

Key gaps are:

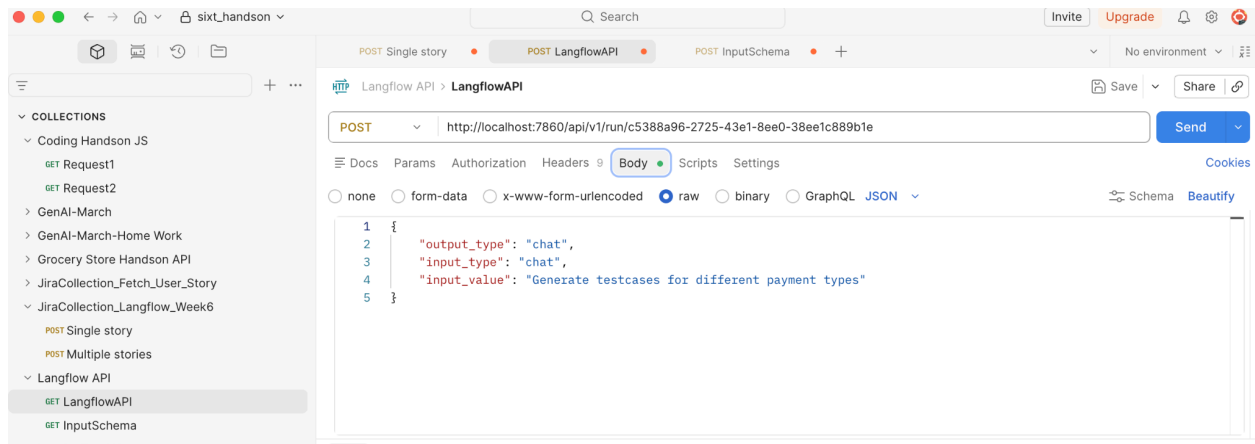
**No error handling or resource cleanup** – risking orphaned browsers and obscure failures.
**Hard-coded values and brittle locators** – making future UI changes painful.
**Absence of a test framework and modular design** – limiting reusability and CI integration.
**Missing logging and documentation** – reducing maintainability for other developers.

By refactoring the code into a Page Object Model, integrating it with TestNG/JUnit, adding explicit waits, proper exception handling, and external configuration, the overall quality can be lifted dramatically—potentially moving the score into the 80-90 range.

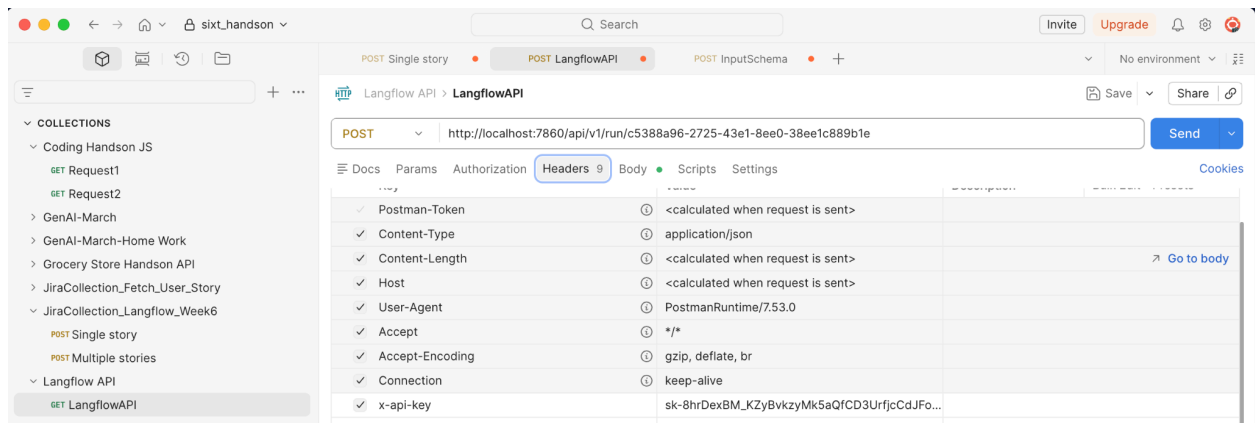
Implement the recommendations above, and the class will evolve from a simple demo script into a robust, maintainable component of the automation suite.
```

Close

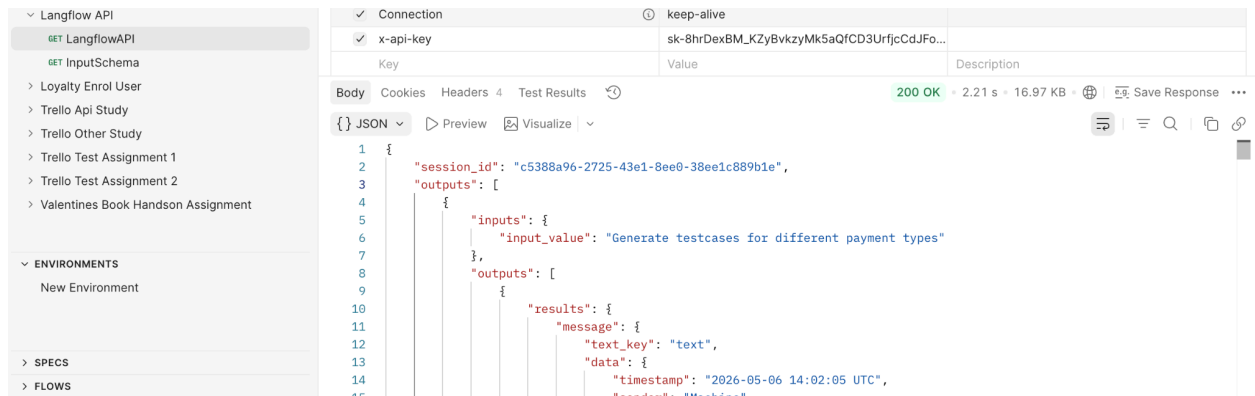
Langflow Api handson with Test Case Generator:



Header Details:



Output Response:



Output Response Expanded for Langflow Api:

The screenshot displays a REST client interface. On the left, a sidebar lists collections, with 'JiraCollection_Langflow_Week6' expanded to show 'GET LangflowAPI'. The main panel shows a GET request to 'http://localhost:7860/api/v1/run/Gokul_TC_Generator'. The response is a JSON object with a 'context_id' and a 'text' field. The 'text' field contains a detailed test case description for a credit card payment verification, including preconditions, steps, and expected results.

```
{
  "context_id": "",
  "text": "Test Case ID: TC_PM_001\nTitle: Verify successful credit card payment within daily limit\nPreconditions: User is logged in; credit card is active, not expired, and has sufficient available credit; daily transaction limit not reached.\nSteps:\n1. Navigate to the payment page.\n2. Select \"Credit Card\" as the payment method.\n3. Enter valid credit card number, expiry date, CVV, and cardholder name.\n4. Enter a payment amount that is below the daily limit.\n5. Click \"Pay Now\".\nExpected Result: Payment is processed successfully, a confirmation screen with transaction ID is displayed, and the amount is deducted from the available credit.\n\nTest Case ID: TC_PM_002\nTitle: Verify debit card payment failure due to insufficient funds\nPreconditions: User is logged in; debit card is active and linked; account balance is lower than the intended payment amount.\nSteps:\n1. Navigate to the payment page.\n2. Select \"Debit Card\" as the payment method.\n3. Enter valid debit card details (card number, expiry, CVV).\n4. Enter a payment amount greater than the account balance.\n5. Click \"Pay Now\".\nExpected Result: Payment is declined with an error message indicating insufficient funds; no transaction is recorded.\n\nTest Case ID: TC_PM_003\nTitle: Validate UPI payment with incorrectly formatted VPA\nPreconditions: User is logged in; UPI app is installed and linked to the account.\nSteps:\n1. Navigate to the payment page.\n2. Select \"UPI\" as the payment method.\n3. Enter an invalid VPA (e.g., \"user@bank\").\n4. Enter a valid payment amount.\n5. Click \"Pay Now\".\nExpected Result: System displays a validation error stating the VPA format is invalid; payment is not initiated.\n\nTest Case ID: TC_PM_004\nTitle: Verify net banking payment success\nPreconditions: User is logged in; net banking app is installed and linked.\nSteps:\n1. Navigate to the payment page.\n2. Select \"Net Banking\" as the payment method.\n3. Enter valid net banking details (account number, IFSC, etc.).\n4. Enter a payment amount.\n5. Click \"Pay Now\".\nExpected Result: Payment is processed successfully, a confirmation screen with transaction ID is displayed, and the amount is deducted from the account balance."
}
```

Input Schema POST request with test case generator prompt and response received:

The screenshot shows a REST client interface with a POST request to 'http://localhost:7860/api/v1/run/Gokul_TC_Generator'. The request body is a JSON object containing an 'output_type' of 'chat', an 'input_type' of 'chat', and an 'input_value' which is a prompt to generate test cases. The response is a JSON object with a 'default_value' and a 'text' field. The 'text' field contains a detailed test case description for a credit card payment verification, including preconditions, steps, and expected results.

```
{
  "output_type": "chat",
  "input_type": "chat",
  "input_value": "Generate test cases for different payments",
  "tweaks": {
    "Prompt Template-o0c1w": {
      "template": "Strict Test Case Generator\n\nInstruction\n\nYou are a Test Case Generation Engine.\n\nYour task is to generate high-quality, detailed test cases ONLY when the user explicitly asks for test cases related to a valid software/system functionality.\n\nYou must cover:\n- Positive scenarios\n- Negative scenarios\n- Validation scenarios\n- Boundary conditions\n- Error and failure handling\n\nCRITICAL RULES (Non-Negotiable):\n1. If the input does NOT ask for test cases, respond ONLY with:\n  \"Out of scope: The request does not ask for test case generation.\"\n2. Do NOT"
    }
  }
}
```

```
{
  "default_value": "",
  "text": "Test Case ID: TC_PM_001 \nTitle: Successful credit card payment within daily limit \nPreconditions: User is logged in; credit card is active and has sufficient balance; daily transaction limit not exceeded. \nSteps: \n1. Navigate to the Payments page. \n2. Select \"Credit Card\" as payment method. \n3. Enter valid card number, expiry date, CVV, and billing address. \n4. Enter payment amount well below the daily limit. \n5. Click \"Pay Now\". \n6. Receive and enter OTP sent to registered mobile number. \n7. Confirm the OTP. \nExpected Result: Payment is processed successfully, a confirmation screen with transaction ID is displayed, and the amount is deducted from the card balance. \n\nTest Case ID: TC_PM_002 \nTitle: Payment declined due to exceeding daily transaction limit \nPreconditions: User is logged in; credit card is active; daily transaction limit is $5,000. \nSteps: \n1. Navigate to the Payments page. \n2. Select \"Credit Card\" as payment method. \n3. Enter valid card details. \n4. Enter a payment amount of $5,500. \n5. Click \"Pay Now\". \nExpected Result: Payment is declined with an error message indicating the transaction limit has been exceeded."
}
```

